

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №4
на тему
**«АСИММЕТРИЧНАЯ КРИПТОГРАФИЯ.
КРИПТОСИСТЕМА МАК-ЭЛИСА»**

БГУИР 6-05-0612-02 098

Выполнил студент группы 353502
ПЕТРОЧЕНКО Михаил Максимович

(дата, подпись студента)

Проверил ассистент кафедры информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Цель работы	3
2	Ход работы	4
2.1	Генерация ключей	6
2.2	Зашифрование и расшифрование текста	7
2.3	Обнаружение искажений	8
	Заключение	10
	Приложение А (обязательное) Листинг программного кода	11
	Приложение Б (обязательное) Блок-схема алгоритма	17

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является изучение принципов построения асимметричных криптосистем на примере криптосистемы Мак-Элиса, стойкость которой основана на сложности декодирования произвольного линейного кода. Задачи включают изучение математических основ криптосистемы – арифметики конечных полей, двоичных кодов Гоппы и алгоритма их декодирования, – проектирование и реализацию программного средства, осуществляющего генерацию ключевой пары, зашифрование открытым ключом и расшифрование закрытым ключом, а также тестирование разработанного программного средства.

Конечным результатом данной лабораторной работы должна быть разработанная программа, осуществляющая зашифрование и расшифрование данных криптосистемой Мак-Элиса как для текста, вводимого с клавиатуры, так и для файлов произвольного содержания.

2 ХОД РАБОТЫ

В ходе выполнения работы было создано программное средство на языке программирования *Rust*. Сторонние библиотеки для алгебры не используются, из внешних зависимостей подключена только библиотека *rand* для генерации случайных чисел.

Элементы поля $GF(2^m)$ представляются многочленами степени меньше m от переменной z с коэффициентами из $GF(2)$ и хранятся как m -битные числа. Сложение элементов – поразрядное исключающее ИЛИ, умножение – умножение многочленов по модулю неприводимого многочлена степени m . Для каждого $m = 2, \dots, 16$ в программе зафиксирован примитивный многочлен, например $z^{10} + z^3 + 1$ для $m = 10$, поэтому корень $\alpha = z$ порождает все ненулевые элементы поля. Это позволяет при создании поля построить таблицы степеней и логарифмов и выполнять умножение и обращение как сложение и вычитание показателей:

$$\alpha^i \cdot \alpha^j = \alpha^{(i+j) \bmod (2^m-1)}, \quad \sqrt{a} = a^{2^{m-1}}. \quad (2.1)$$

При построении таблиц проверяется, что степени α пробегает все ненулевые элементы, то есть что многочлен действительно примитивен.

Двоичный код Гоппы задаётся носителем $L = (\alpha_1, \dots, \alpha_n)$ из n различных элементов $GF(2^m)$ и неприводимым многочленом Гоппы $g(x)$ степени t с коэффициентами из $GF(2^m)$. Код состоит из двоичных векторов c длины n , для которых

$$S_c(x) = \sum_{i=1}^n \frac{c_i}{x - \alpha_i} \equiv 0 \pmod{g(x)}. \quad (2.2)$$

Такой код имеет размерность $k \geq n - mt$ и минимальное расстояние не менее $2t + 1$, то есть исправляет до t ошибок. Проверочная матрица кода составляется из элементов $\alpha_i^j / g(\alpha_i)$, $j = 0, \dots, t - 1$; каждый элемент раскладывается на m бит, что даёт двоичную матрицу H размера $mt \times n$. Приведением H методом Гаусса к виду $[A \mid I]$ получается порождающая матрица $G = [I_k \mid A^T]$ в систематическом виде, при котором первые k бит кодового слова совпадают с сообщением. Если при приведении приходится переставлять столбцы, так же переставляется и носитель L , иначе матрица G и декодер описывали бы разные коды.

Многочлен Гоппы выбирается случайно: генерируются случайные унитарные многочлены степени t , пока очередной не пройдёт тест Бен-Ора, согласно которому многочлен g степени t над $GF(q)$ неприводим тогда и только тогда, когда $\gcd(g(x), x^{q^i} - x) = 1$ для всех $i = 1, \dots, \lfloor t/2 \rfloor$. Неприводимым оказывается примерно один многочлен из t , поэтому требуется в среднем около t попыток.

Ключевая пара строится из случайного кода Гоппы, случайной невырожденной матрицы S размера $k \times k$ и случайной перестановки P длины n :

$$G' = S \cdot G \cdot P. \quad (2.3)$$

Открытым ключом являются матрица G' и число t , закрытым – носитель L , многочлен $g(x)$, матрица S^{-1} и перестановка P^{-1} . Матрица S перемешивает базис кода, а P – координаты, поэтому G' внешне неотличима от порождающей матрицы случайного линейного кода. Невырожденная матрица находится перебором случайных матриц: обратимой оказывается около 29 % из них, а обратная вычисляется тем же методом Гаусса – Жордана. Параметры, предусмотренные в программе, приведены в таблице 2.1.

Таблица 2.1 – Наборы параметров криптосистемы

Набор	m	n	t	k	Размер G' , байт
toy	4	16	2	8	16
small	8	256	8	192	6144
original (1978)	10	1024	50	524	67 072

Зашифрование блока m длины k бит выполняется по формуле

$$c = m \cdot G' + e, \quad (2.4)$$

где e – случайный вектор длины n ровно с t единицами. Благодаря случайному вектору ошибок зашифрование вероятностное: одно и то же сообщение при каждом зашифровании даёт новый шифротекст.

Расшифрование начинается со снятия перестановки: $c \cdot P^{-1} = (mS) \cdot G + e \cdot P^{-1}$. Перестановка не меняет число единиц, поэтому полученный вектор u отличается от кодового слова mSG ровно в t позициях, и их находит алгоритм Паттерсона. Вычисляется синдром $S(x)$; если он нулевой, ошибок нет. Иначе ищется многочлен локаторов ошибок $\sigma(x) = \prod_{i \in E} (x - \alpha_i)$, корни которого – элементы носителя в позициях ошибок E . Он удовлетворяет ключевому уравнению $S(x)\sigma(x) \equiv \sigma'(x) \pmod{g(x)}$. В характеристике 2 многочлен раскладывается как $\sigma = a^2 + xb^2$, откуда $\sigma' = b^2$, и уравнение сводится к

$$a(x) \equiv b(x)\tau(x) \pmod{g(x)}, \quad \tau(x) = \sqrt{S(x)^{-1} + x} \pmod{g(x)}. \quad (2.5)$$

Многочлены a и b находятся расширенным алгоритмом Евклида для g и τ , остановленным на первом остатке степени не выше $\lfloor t/2 \rfloor$; при этом степень b не превышает $\lfloor (t-1)/2 \rfloor$, и $\deg \sigma \leq t$. Квадратный корень по модулю неприводимого g вычисляется как $\sqrt{p} = p_0 + \sqrt{x} \cdot p_1$, где $p = p_0^2 + xp_1^2$, а $\sqrt{x} \pmod{g}$ находится один раз при создании ключа. Позиции ошибок определяются подстановкой всех элементов носителя в $\sigma(x)$, после чего соответствующие биты у

инвертируются. Если число найденных корней не равно степени σ или исправленный вектор не является кодовым словом, ошибок больше t , и блок считается искажённым. Первые k бит исправленного слова равны mS , и сообщение восстанавливается умножением на S^{-1} . Блок-схема алгоритма приведена в приложении Б.

Сообщение переводится в последовательность бит (младший бит каждого байта первым), дополняется одной единицей и нулями до длины, кратной k , и разбивается на блоки по k бит. Дополнение добавляется всегда, поэтому при расшифровании оно снимается однозначно: отбрасываются последняя единица и нули за ней, причём единица должна стоять на границе байта в пределах последнего блока. Каждый шифроблок из n бит записывается в $\lceil n/8 \rceil$ байт; для параметров 1978 года это 128 байт на каждые 524 бита сообщения, то есть шифротекст примерно вдвое длиннее открытого текста.

Перед расшифрованием проверяется, что длина шифротекста ненулевая и кратна длине шифроблока, а биты за пределами n в последнем байте каждого блока нулевые. Если какой-либо блок не декодируется или дополнение оказывается некорректным, программа сообщает об искажении шифротекста.

Исходный код криптосистемы Мак-Элиса приведён в приложении А в листинге А.1.

2.1 Генерация ключей

При запуске программа генерирует ключевую пару с параметрами из статьи Мак-Элиса 1978 года: $m = 10$, $n = 1024$, $t = 50$, $k = 524$. Генерация выполняется в несколько этапов. Сначала строится поле $\text{GF}(2^{10})$ по примитивному многочлену $z^{10} + z^3 + 1$ и выбирается носитель из 1024 элементов; так как $n = 2^m$, в него входят все элементы поля, взятые в случайном порядке. Затем подбирается неприводимый многочлен Гоппы степени 50, для чего в среднем требуется около 50 проверок тестом Бен-Ора. Проверочная матрица H имеет размер 500×1024 , её ранг равен 500, и приведение к систематическому виду даёт порождающую матрицу G размера 524×1024 , то есть $k = n - mt$. Наконец выбираются случайная невырожденная матрица S размера 524×524 и перестановка P длины 1024, и вычисляется открытая матрица $G' = SGP$. Вся генерация заняла около 12 мс.

Меню программы и сведения о ключах представлены на рисунке 2.1. Открытый ключ состоит из параметров n , k , t и матрицы G' , которая занимает $524 \cdot 1024$ бит, или 67 072 байта. В закрытом ключе программа показывает поле, многочлен Гоппы $g(x)$, коэффициенты которого записаны степенями примитивного элемента α , носитель L , матрицу S^{-1} размером $524 \cdot 524$ бит, или 34 322 байта, и перестановку P^{-1} . Носитель и перестановка содержат по 1024 элемента, поэтому программа выводит только первые 64 из них и число оставшихся, а матрицы – только размеры. Новую ключевую пару можно сгенерировать командой `g`.

```

Generating keys for m = 10, n = 1024, t = 50...
Done in 12.15ms

McEliece cryptosystem - input/output: text, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): k

Public key
  n, k, t      : 1024, 524, 50
  G' = S·G·P   : 524 × 1024, 67072 bytes
Private key
  field        : GF(210) = GF(2)[z]/(z10 + z3 + 1)
  g(x)         : x50 + α402·x49 + α727·x48 + α18·x47 + α529·x46 + α1012·x45 +
  α658·x44 + α464·x43 + α568·x42 + α710·x41 + α910·x40 + α949·x39 + α339·x38 +
  α418·x37 + α735·x36 + α103·x35 + α899·x34 + α374·x33 + α254·x32 + α696·x31 +
  α334·x30 + α539·x29 + α734·x28 + α912·x27 + α877·x26 + α638·x25 + α380·x24 +
  α190·x23 + α885·x22 + α411·x21 + α850·x20 + α939·x19 + α818·x18 + α61·x17 +
  α464·x16 + α676·x15 + α246·x14 + α601·x13 + α539·x12 + α107·x11 + α530·x10 +
  α1014·x9 + α832·x8 + α1001·x7 + α840·x6 + α209·x5 + α197·x4 + α251·x3 +
  α731·x2 + α941·x + α390
  support L    : α977, α102, α883, α198, α827, α880, α340, α829, α569, α719, α240,
  α546, α892, α297, α425, α901, α104, α836, α249, α414, α725, α736, α428, α103,
  α720, α791, α850, α404, α413, α779, α309, α170, α782, α355, α421, α682, α818,
  α463, α982, α391, α377, α574, α911, α873, α959, α472, α369, α516, α341, α279,
  α746, α56, α512, α974, α831, α346, α914, α878, α283, α603, α895, α490, α378,
  α255, ... (960 more)
  S-1       : 524 × 524, 34322 bytes
  P-1       : 969, 86, 987, 286, 144, 797, 905, 620, 725, 168, 187, 808, 621, 266, 457, 847,
  579, 927, 21, 979, 339, 855, 43, 492, 774, 1004, 888, 644, 839, 607, 878, 971, 145, 822,
  1021, 352, 915, 319, 514, 720, 106, 98, 530, 633, 91, 447, 395, 54, 597, 887, 683, 181, 112,
  646, 513, 781, 265, 140, 823, 864, 336, 75, 1, 388, ... (960 more)

```

Рисунок 2.1 – Меню программы и ключевая пара с параметрами 1978 года

2.2 Зашифрование и расшифрование текста

Результат работы программы для текста, вводимого с клавиатуры, представлен на рисунке 2.2. Сообщение «Привет, Мак-Элис!» в кодировке UTF-8 занимает 30 байт, то есть 240 бит. Вместе с единичным битом дополнения и 283 нулями оно образует ровно один блок из 524 бит, который умножается на G' и складывается с вектором ошибок. Шифроблок из 1024 бит записывается в 128 байт и выводится в шестнадцатеричном виде.

Повторное зашифрование того же сообщения тем же ключом даёт другой шифротекст, так как каждый раз выбирается новый вектор ошибок. При этом два шифротекста заметно похожи: их сумма равна $mG' + e_1 + mG' + e_2 = e_1 + e_2$ и содержит не больше $2t = 100$ единиц. В показанном примере шифротексты различаются в 98 битах, затрагивающих 70 байт из 128, а остальные байты совпадают. Поэтому одно и то же сообщение в базовой схеме нельзя шифровать повторно: сумма шифротекстов раскрывает позиции ошибок.

При расшифровании шифротекст вводится в шестнадцатеричном виде. Программа проверяет его длину, снимает перестановку, исправляет 50 ошибок декодером Паттерсона, умножает первые 524 бита кодового слова на S^{-1} и снимает дополнение, после чего выводит исходный текст и его длину. Оба шифротекста расшифровываются в исходное сообщение.

```

McEliece cryptosystem - input/output: text, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): 1

Plaintext: Привет, Мак-Элис!
mode      : encryption
plaintext : 30 bytes
encrypted : 29548F7A750D8A3750F3DB55B59337078C78834EBA0599E7D6200476AB58B382B1682229467370D041
1F6EA7D970C50E7903C7616164F9C3352D4D67B0E20D29ED9B42555C037CC8ED33DB7B0A81F810C383F4A15EF83B
0E850D2E9BBAE9983845C5FEDD1E3F012F129A26FFF16A383EA29AF0A14F83886AE202350274F682D0 (128
bytes)

McEliece cryptosystem - input/output: text, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): 1

Plaintext: Привет, Мак-Элис!
mode      : encryption
plaintext : 30 bytes
encrypted : 295E871AFD0D823710F3DE57F49337078C58834EBA0599E3F2300674ABD82682A068A3205473704081
1F6CA2D950C50E7903DF616164F1C1B54D4D6FF0E20D69ED9962571C037CC8CD33DA7B0B83F8184383B5A97E883B
0E010D2E9FBAE9D82847C576DD1E3F012F329E06FFF90B3036E39AF2A14F9F806EE202352254F7C2D0 (128
bytes)

McEliece cryptosystem - input/output: text, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): 2

Ciphertext (hex): 29548F7A750D8A3750F3DB55B59337078C78834EBA0599E7D6200476AB58B382B1682229467370
D0411F6EA7D970C50E7903C7616164F9C3352D4D67B0E20D29ED9B42555C037CC8ED33DB7B0A81F810C383F4A15EF8
3B0E850D2E9BBAE9983845C5FEDD1E3F012F129A26FFF16A383EA29AF0A14F83886AE202350274F682D0
mode      : decryption
encrypted : 128 bytes
decrypted : Привет, Мак-Элис! (30 bytes)

```

Рисунок 2.2 – Зашифрование и расшифрование текста

2.3 Обнаружение искажений

Реакция программы на искажённый шифротекст показана на рисунке 2.3. В качестве исходного взят первый шифротекст из рисунка 2.2. В первом случае в нём изменён младший бит 21-го байта, ВА заменено на ВВ. К 50 ошибкам, внесённым при зашифровании, добавилась 51-я, и декодер, исправляющий не более $t = 50$ ошибок, отвергает блок: найденный многочлен локаторов не имеет нужного числа корней среди элементов носителя. Если бы изменённый бит совпал с одной из позиций вектора ошибок, ошибок осталось бы 49 и блок расшифровался бы верно. Изменение шифротекста, таким образом, обнаруживается не всегда, и различие в реакции декодера позволяет по отдельным битам выяснять позиции ошибок, поэтому базовая схема не защищена от атак с подобранным шифротекстом.

Во втором случае шифротекст укорочен на один байт до 127 байт. Он отвергается ещё до расшифрования, поскольку его длина не кратна длине шифроблока в 128 байт.

```

McEliece cryptosystem - input/output: text, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): 2

Ciphertext (hex): 29548F7A750D8A3750F3DB55B59337078C78834EBB0599E7D6200476AB58B382B1682229467370
D0411F6EA7D970C50E7903C76164F9C3352D4D67B0E20D29ED9B42555C037CC8ED33DB7B0A81F810C383F4A15EF8
3B0E850D2E9BBAE9983845C5FEDD1E3F012F129A26FFF16A383EA29AF0A14F83886AE202350274F682D0
Error: Block corrupted

McEliece cryptosystem - input/output: text, n = 1024, k = 524, t = 50
1) encrypt          2) decrypt
3) trace one block
Choice (or 'm' to switch text/file mode, 'k' to see keys, 'g' to generate keys, 'q' to quit): 2

Ciphertext (hex): 29548F7A750D8A3750F3DB55B59337078C78834EBA0599E7D6200476AB58B382B1682229467370
D0411F6EA7D970C50E7903C76164F9C3352D4D67B0E20D29ED9B42555C037CC8ED33DB7B0A81F810C383F4A15EF8
3B0E850D2E9BBAE9983845C5FEDD1E3F012F129A26FFF16A383EA29AF0A14F83886AE202350274F682
Error: Invalid message length

```

Рисунок 2.3 – Расшифрование искажённого шифротекста

Помимо проверки вручную, корректность реализации подтверждена 51 модульным тестом, покрывающим все уровни программы. Умножение по таблицам сверено с прямым умножением многочленов для всех пар элементов $GF(2^4)$ и $GF(2^8)$, тест неприводимости – с точной формулой числа неприводимых многочленов. Для кода Гоппы проверено равенство $GH^T = 0$, а для кода над $GF(2^4)$ с $n = 16$ и $t = 2$ перебором всех кодовых слов подтверждено минимальное расстояние не менее $2t + 1 = 5$. Декодер проверен на случайных векторах ошибок веса от 0 до t для нескольких наборов параметров, включая $n = 1024, t = 50$; при $t + 1$ ошибках он никогда не возвращает исходное кодовое слово. Криптосистема в целом проверена зашифрованием и расшифрованием сообщений длиной от 0 до 257 байт, в том числе граничных длин, текстов на кириллице и файлов, а также искажённых шифротекстов. Во всех случаях исходное сообщение восстанавливается без искажений, а некорректный шифротекст отвергается.

Таким образом криптосистема Мак-Элиса была изучена, реализована и протестирована в полном объёме как для текста, вводимого с клавиатуры, так и для файлов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены принципы построения асимметричных криптосистем на примере криптосистемы Мак-Элиса. Было разработано программное средство на языке *Rust*, реализующее арифметику конечных полей $GF(2^m)$, многочленов над ними и матриц над $GF(2)$, построение двоичных кодов Гоппы и их декодирование алгоритмом Паттерсона, а на их основе – генерацию ключевой пары, зашифрование открытым ключом и расшифрование закрытым ключом над текстом, вводимым с клавиатуры, либо над файлами произвольного содержания.

Тестирование показало, что расшифрование восстанавливает исходное сообщение без искажений при любой длине, а искажение шифротекста, вносящее больше t ошибок в блок, обнаруживается. Работа показала основные особенности криптосистемы. Её стойкость основана не на теории чисел, а на NP-трудной задаче декодирования произвольного линейного кода, для которой неизвестны эффективные алгоритмы, в том числе квантовые, поэтому криптосистема относится к постквантовым. Зашифрование сводится к умножению вектора на матрицу и сложению с вектором ошибок и выполняется очень быстро, а зашифрование вероятностное благодаря случайному вектору ошибок.

Отдельно следует отметить отличия от изученной ранее криптосистемы Рабина: преобразования выполняются не над большими целыми числами, а над битовыми векторами и многочленами над конечными полями; открытый ключ очень велик и для параметров 1978 года занимает 67 072 байта против 384 байт модуля Рабина; шифротекст примерно вдвое длиннее открытого текста; наконец, для однозначного расшифрования не требуется избыточность, так как декодер восстанавливает единственное кодовое слово.

Все задачи лабораторной работы выполнены в полном объёме.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

Листинг А.1 – Реализация криптосистемы Мак-Элиса

```
use super::goppa::Goppa;
use super::matrix::{BitVec, Matrix, Permutation};
use rand::rngs::OsRng;
use rand::{Rng, SeedableRng, rngs::StdRng};

#[derive(Clone, Copy, Debug, PartialEq, Eq)]
pub struct Params {
    pub m: u32,
    pub n: usize,
    pub t: usize,
}

impl Params {
    pub const TOY: Params = Params { m: 4, n: 16, t: 2 };
    pub const SMALL: Params = Params { m: 8, n: 256, t: 8 };
    pub const ORIGINAL: Params = Params {
        m: 10,
        n: 1024,
        t: 50,
    };
}

#[derive(Clone)]
pub struct PublicKey {
    matrix: Matrix,
    t: usize,
}

impl PublicKey {
    pub fn matrix(&self) -> &Matrix {
        return &self.matrix;
    }

    pub fn n(&self) -> usize {
        return self.matrix.col_count();
    }

    pub fn k(&self) -> usize {
        return self.matrix.row_count();
    }

    pub fn t(&self) -> usize {
        return self.t;
    }

    fn block_bytes(&self) -> usize {
        return self.n().div_ceil(8);
    }
}

pub struct PrivateKey {
    public: PublicKey,
    code: Goppa,
    s_inv: Matrix,
    p_inv: Permutation,
}
```

```

impl PrivateKey {
    pub fn public(&self) -> &PublicKey {
        return &self.public;
    }

    pub fn code(&self) -> &Goppa {
        return &self.code;
    }

    pub fn s_inv(&self) -> &Matrix {
        return &self.s_inv;
    }

    pub fn p_inv(&self) -> &Permutation {
        return &self.p_inv;
    }
}

pub struct McEliece {}

impl McEliece {
    pub fn keygen(params: Params) -> Result<(PrivateKey, PublicKey), String>
    {
        let mut rng = StdRng::from_rng(&mut OsRng).unwrap();

        return Self::keygen_with_rng(params, &mut rng);
    }

    pub fn keygen_with_rng<R: Rng>(
        params: Params,
        rng: &mut R,
    ) -> Result<(PrivateKey, PublicKey), String> {
        let code = Goppa::generate(params.m, params.t, rng)?;
        let (s, s_inv) = Matrix::random_invertible(code.k(), rng);
        let p = Permutation::random(code.n(), rng);

        let public = PublicKey {
            matrix: s.mul(code.generator()).permute_columns(&p),
            t: code.t(),
        };
        let private = PrivateKey {
            public: public.clone(),
            code,
            s_inv,
            p_inv: p.inverse(),
        };

        return Ok((private, public));
    }

    pub fn encrypt_block<R: Rng>(message: &BitVec, key: &PublicKey, rng: &mut
R) -> BitVec {
        let e = BitVec::random_with_weight(key.n(), key.t(), rng);

        return key.matrix.vec_mul(message).xor(&e);
    }

    pub fn decrypt_block(cipher: &BitVec, key: &PrivateKey) -> Result<BitVec,
String> {
        let codeword = key.code.decode(&key.p_inv.apply(cipher))?;
        let message_s = key.code.message(&codeword);

```

```

        return Ok(key.s_inv.vec_mul(&message_s));
    }

    pub fn encrypt(plain_bytes: Vec<u8>, key: &PublicKey) -> Vec<u8> {
        let mut rng = StdRng::from_rng(&mut OsRng).unwrap();

        return Self::encrypt_with_rng(plain_bytes, key, &mut rng);
    }

    pub fn encrypt_with_rng<R: Rng>(plain_bytes: Vec<u8>, key: &PublicKey,
rng: &mut R) -> Vec<u8> {
        let k = key.k();
        let data_len = plain_bytes.len() * 8;
        let blocks = data_len / k + 1;

        let data = BitVec::from_bytes(&plain_bytes, data_len).unwrap();
        let mut padded = BitVec::zeros(blocks * k);
        for i in data.iter_ones() {
            padded.set(i, true);
        }
        padded.set(data_len, true);

        let mut result = Vec::with_capacity(blocks * key.block_bytes());
        for b in 0..blocks {
            let message = padded.slice(b * k..(b + 1) * k);
            result.extend(Self::encrypt_block(&message, key,
rng).to_bytes());
        }

        return result;
    }

    pub fn decrypt(encrypted_bytes: Vec<u8>, key: &PrivateKey) ->
Result<Vec<u8>, String> {
        let (n, k) = (key.public.n(), key.public.k());
        let block_bytes = key.public.block_bytes();
        if encrypted_bytes.is_empty() || !
encrypted_bytes.len().is_multiple_of(block_bytes) {
            return Err("Invalid message length".to_owned());
        }

        let blocks = encrypted_bytes.len() / block_bytes;
        let mut padded = BitVec::zeros(blocks * k);
        for (b, chunk) in encrypted_bytes.chunks(block_bytes).enumerate() {
            let cipher = BitVec::from_bytes(chunk, n).map_err(|_| "Block
corrupted".to_owned())?;
            let message =
                Self::decrypt_block(&cipher, key).map_err(|_| "Block
corrupted".to_owned())?;
            for i in message.iter_ones() {
                padded.set(b * k + i, true);
            }
        }

        match padded.last_one() {
            Some(pos) if pos.is_multiple_of(8) && pos + k >= padded.len() =>
{
                return Ok(padded.prefix(pos).to_bytes());
            }
            _ => return Err("Invalid padding".to_owned()),
        }
    }
}

```

```

#[cfg(test)]
mod tests {
    use super::*;

    fn rng() -> StdRng {
        return StdRng::seed_from_u64(4);
    }

    #[test]
    fn keys_fit_together() {
        let mut rng = rng();
        let (private, public) = McEliece::keygen_with_rng(Params::SMALL, &mut
rng).unwrap();
        let code = private.code();
        assert_eq!(
            (public.n(), public.k(), public.t()),
            (code.n(), code.k(), code.t())
        );

        let s = private.s_inv().inverse().unwrap();
        let p = private.p_inv().inverse();
        assert_eq!(
            public.matrix(),
            &s.mul(code.generator()).permute_columns(&p)
        );

        assert_ne!(
            public.matrix().columns(0..public.k()),
            Matrix::identity(public.k())
        );
    }

    #[test]
    fn block_round_trip() {
        let mut rng = rng();
        for params in [Params::TOY, Params::SMALL, Params::ORIGINAL] {
            let (private, public) = McEliece::keygen_with_rng(params, &mut
rng).unwrap();
            for _ in 0..20 {
                let message = BitVec::random(public.k(), &mut rng);
                let cipher = McEliece::encrypt_block(&message, &public, &mut
rng);
                assert_eq!(cipher.len(), public.n());
                assert_eq!(McEliece::decrypt_block(&cipher,
&private).unwrap(), message);
            }
        }
    }

    #[test]
    fn error_vector_has_weight_t() {
        let mut rng = rng();
        let (_, public) = McEliece::keygen_with_rng(Params::SMALL, &mut
rng).unwrap();
        let zero = BitVec::zeros(public.k());
        for _ in 0..20 {
            assert_eq!(
                McEliece::encrypt_block(&zero, &public, &mut rng).weight(),
                public.t()
            );
        }
    }
}

```

```

#[test]
fn bytes_round_trip() {
    let mut rng = rng();
    for params in [Params::TOY, Params::SMALL] {
        let (private, public) = McEliece::keygen_with_rng(params, &mut
rng).unwrap();
        let k = public.k();
        for len in [0, 1, 2, 3, k / 8, k / 8 + 1, 2 * k / 8, 100, 257] {
            let plain: Vec<u8> = (0..len).map(|_| rng.r#gen()).collect();
            let cipher = McEliece::encrypt_with_rng(plain.clone(),
&public, &mut rng);
            let blocks = (len * 8) / k + 1;
            assert_eq!(cipher.len(), blocks * public.n().div_ceil(8));
            assert_eq!(McEliece::decrypt(cipher, &private).unwrap(),
plain);
        }
    }
}

#[test]
fn original_parameters_round_trip() {
    let mut rng = rng();
    let (private, public) = McEliece::keygen_with_rng(Params::ORIGINAL,
&mut rng).unwrap();
    let plain = "Криптосистема Мак-Элиса, 1978".as_bytes().to_vec();
    let cipher = McEliece::encrypt(plain.clone(), &public);
    assert_eq!(McEliece::decrypt(cipher, &private).unwrap(), plain);
}

#[test]
fn encryption_is_randomized() {
    let mut rng = rng();
    let (private, public) = McEliece::keygen_with_rng(Params::SMALL, &mut
rng).unwrap();
    let plain = b"same text".to_vec();
    let first = McEliece::encrypt_with_rng(plain.clone(), &public, &mut
rng);
    let second = McEliece::encrypt_with_rng(plain.clone(), &public, &mut
rng);
    assert_ne!(first, second);
    assert_eq!(McEliece::decrypt(first, &private).unwrap(), plain);
    assert_eq!(McEliece::decrypt(second, &private).unwrap(), plain);
}

#[test]
fn rejects_malformed_ciphertexts() {
    let mut rng = rng();
    let (private, public) = McEliece::keygen_with_rng(Params::SMALL, &mut
rng).unwrap();
    let cipher = McEliece::encrypt_with_rng(b"hello".to_vec(), &public,
&mut rng);

    assert!(McEliece::decrypt(vec![], &private).is_err());
    assert!(McEliece::decrypt(cipher[1..].to_vec(), &private).is_err());

    let zero_block = McEliece::encrypt_block(&BitVec::zeros(public.k()),
&public, &mut rng);
    assert!(McEliece::decrypt(zero_block.to_bytes(), &private).is_err());

    let (other, _) = McEliece::keygen_with_rng(Params::SMALL, &mut
rng).unwrap();
    assert_ne!(

```

```

        McEliece::decrypt(cipher, &other).ok(),
        Some(b"hello".to_vec())
    );
}

#[test]
fn rejects_bits_past_n() {
    let mut rng = rng();
    let params = Params { m: 5, n: 30, t: 3 };
    let (private, public) = McEliece::keygen_with_rng(params, &mut
rng).unwrap();
    let mut cipher = McEliece::encrypt_with_rng(b"x".to_vec(), &public,
&mut rng);
    assert_eq!(McEliece::decrypt(cipher.clone(), &private).unwrap(),
b"x");
    cipher[3] |= 0x80;
    assert!(McEliece::decrypt(cipher, &private).is_err());
}

#[test]
fn one_flipped_bit_is_fatal_or_absorbed() {
    let mut rng = rng();
    let (private, public) = McEliece::keygen_with_rng(Params::TOY, &mut
rng).unwrap();
    let message = BitVec::random(public.k(), &mut rng);
    let cipher = McEliece::encrypt_block(&message, &public, &mut rng);
    let error_positions: Vec<usize> = cipher
        .xor(&public.matrix().vec_mul(&message))
        .iter_ones()
        .collect();

    for i in 0..public.n() {
        let mut tampered = cipher.clone();
        tampered.flip(i);
        let result = McEliece::decrypt_block(&tampered, &private);
        if error_positions.contains(&i) {
            assert_eq!(result.unwrap(), message);
        } else {
            assert_ne!(result.ok(), Some(message.clone()));
        }
    }
}
}

```

ПРИЛОЖЕНИЕ Б **(обязательное)** **Блок-схема алгоритма**

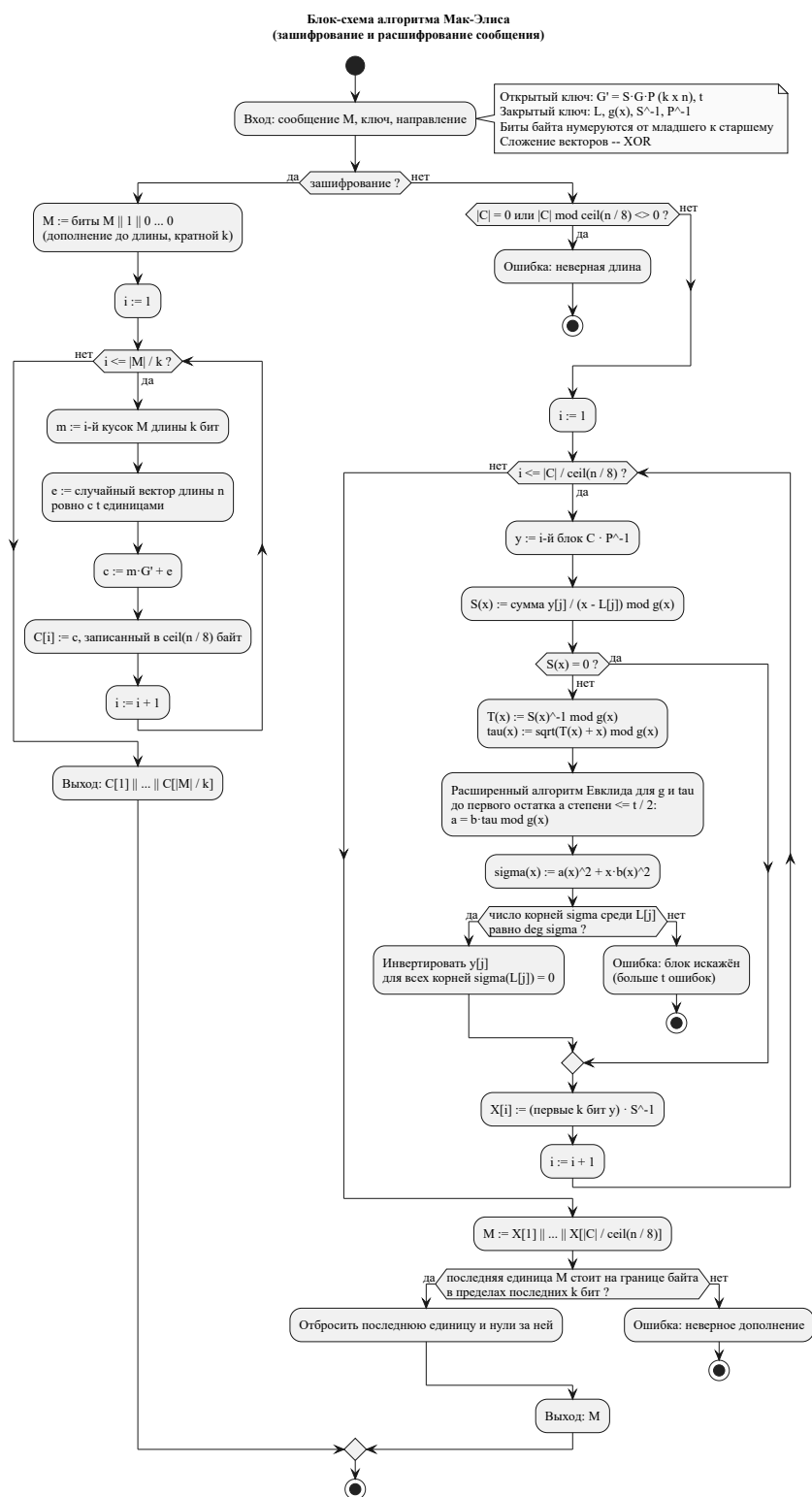


Рисунок Б.1 – Блок-схема алгоритма